



VIT[®]
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)
CHENNAI

OLMNS505 - Exploratory Data Analysis (EDA)

Module 1.

Lecture Notes

Introduction- Packages and Library- Data types – Basic Operators – R objects Vectors- Lists – Arrays – Matrix- Factors – Data frame – R file formats – Import Export File – Data Visualization : Packages - Plots



Module1:

1.1.Introduction to R

R is a programming language and software environment widely used for statistical analysis, graphical representation, and reporting. Developed by Ross Ihaka and Robert Gentleman at the University of Auckland, New Zealand, it has become a cornerstone in the field of data science and statistics. R is open-source and free, making it accessible to a broad range of users from academics to industry professionals.

Features of R

- **Statistical Analysis:** R provides an extensive array of tools for various statistical analyses.
- **Graphics:** It offers powerful graphics capabilities for data visualization, crucial for data analysis and interpretation.
- **Packages:** The Comprehensive R Archive Network (CRAN) hosts thousands of packages for diverse needs in data analysis, visualization, and reporting.
- **Platform Independent:** R is available on various platforms like Windows, Mac, and Linux.
- **Programming Language:** As a programming language, R allows users to write their functions, models, or methods.
- **Community Support:** R has a strong and active community, which is an invaluable resource for learning and problem-solving.

Evolution of R

R is an implementation of the S programming language, which was created to turn ideas into software, quickly and faithfully. The evolution of R from S signifies a focus on making statistical computing more user-friendly and applicable to a wider range of statistical techniques. It is continuously updated and improved by the core R development team and the broader community.

R in Data Science

In the realm of data science, R is particularly esteemed for its versatility in data manipulation, analysis, and visualization. It's widely used for:



- **Data Cleaning and Preparation:** R is excellent for transforming raw data into a form suitable for analysis.
- **Statistical Analysis:** With its roots in statistics, R provides a robust environment for applying statistical techniques.
- **Data Visualization:** From basic plots to intricate interactive visualizations, R's capabilities are extensive.
- **Machine Learning:** R has packages for implementing machine learning algorithms, making it a good choice for predictive analytics.

R versus Other Statistical Tools

R often gets compared to other tools like Python, MATLAB, or SAS. While each tool has its strengths, R is particularly renowned for its statistical analysis capabilities and graphical models. The choice between these tools usually depends on the specific requirements of the project, the user's familiarity with the language, and the type of analysis being conducted.

1.2.Installation and Setup of R

Downloading R

To begin using R, the first step is to install the R software. R can be downloaded from the Comprehensive R Archive Network (CRAN) website. Here's how to do it:

1. **Visit CRAN:** Go to the CRAN website (<https://cran.r-project.org/>) and find the download link for your operating system (Windows, Mac, or Linux).
2. **Select Version:** Choose the latest version of R for download. It's important to keep R updated for better functionality and security.
3. **Download and Install:** Follow the instructions on the site for installation. The process is straightforward, typically involving accepting the default settings.

Setting Up RStudio

While R can be used in its basic form, RStudio, an integrated development environment (IDE) for R, is highly recommended for a more user-friendly experience. RStudio provides a console, syntax-highlighting editor, and tools for plotting, history, debugging, and workspace management.

1. **Download RStudio:** Go to the RStudio website (<https://www.rstudio.com/>) and download the free version of RStudio Desktop.



VIT[®]
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)
CHENNAI

2. **Installation:** Install RStudio after installing R. RStudio detects the R version automatically.
3. **Explore RStudio Interface:** Familiarize yourself with the various panes – script editor, console, environment/history, and files/plots/packages/help.

Basic Configuration

After installation, some basic configuration steps can help streamline the experience:

- **Set Working Directory:** Use `setwd("path/to/directory")` to set your default working directory where R will read and save files.
- **Update Packages:** Regularly update your installed packages with `update.packages()` command.

Testing the Installation

To ensure R and RStudio are correctly installed and configured:

1. **Open RStudio:** Launch the application.
2. **Run a Test Script:** In the console or script editor, type a simple command like `print("Hello, R!")` and run it.
3. **Check for Output:** If R is working properly, it should display "Hello, R!" in the console.

Learning Basics of R Programming

Initially, it's beneficial to learn basic R commands and script writing. Some key areas to start with include:

- **Data Types and Structures:** Understand the basic data types (numeric, character, logical) and structures (vectors, lists, data frames).
- **Basic Operations:** Learn how to perform simple operations and use basic functions.
- **Help System:** Use the help system in R and RStudio for assistance

1.3.The R Interface

Understanding the R Interface



R provides a user interface designed for data analysis and statistical computing. When you first open R, you are greeted with a console where you can type and execute commands. However, for a more enhanced experience, using R through RStudio, a popular IDE for R, is recommended. RStudio offers an organized workspace with various panes and tools, improving the user experience and productivity.

Components of the RStudio Interface

RStudio's interface is divided into several key areas:

1. **Console Pane:** This is where R commands are entered and executed. It's also where results are displayed.
2. **Script Editor:** Here, you can write, edit, and save R scripts. Scripts are simply text files containing a series of R commands.
3. **Environment/History Pane:** This pane shows the current workspace (variables, data sets) and command history. It allows for easy tracking and management of objects created during the R session.
4. **Files/Plots/Packages/Help Pane:** This multifunctional pane lets you navigate files, visualize plots, manage packages, and access R documentation.

Basic R Commands and Syntax

R commands are instructions you give to the R interpreter. Here are some fundamental aspects:

- **Case Sensitivity:** R is case-sensitive. This means **Variable**, **variable**, and **VARIABLE** are different objects in R.
- **Assignment Operators:** You can use `<-` or `=` for assignments, e.g., `x <- 10` or `x = 10`.
- **Comments:** Use `#` to add comments. Anything following `#` on a line is ignored by R.
- **Running Commands:** Type commands in the console or script editor and press Enter (in the console) or Ctrl + Enter (in the script editor) to run them.

1.4.Executing Scripts

Scripts in R are executed sequentially. You can run an entire script or select specific lines to execute. This is particularly useful for running complex analyses or reproducing results.



Basic Data Analysis Flow: A typical R session for data analysis might involve the following steps:

1. **Importing Data:** Load data into R from various sources like CSV files, databases, or web APIs.
2. **Data Wrangling:** Manipulate and prepare data for analysis (e.g., sorting, filtering, transforming).
3. **Analysis:** Perform statistical analyses or apply models to the data.
4. **Visualization:** Create plots and charts to understand and present the data.
5. **Exporting Results:** Save results, whether data, plots, or summary reports.

Saving and Loading Work

In RStudio, you can save your entire workspace (all objects in the current session) and reload it later. This is helpful in long-term projects where you want to preserve your work between sessions.

Run R Programming on Your Computer

You will find the easiest way to run R programming on your system (Windows) in this section.

Run R Programming in Windows

1. Go to official site of R programming(<https://www.r-project.org/>)
2. Click on the CRAN link on the left sidebar
3. Select a mirror
4. Click "Download R for Windows"
5. Click on the link that downloads the base distribution
6. Run the file and follow the steps in the instructions to install R.

1.5.History of R Programming

R is an open-source implementation of the S programming language combined with lexical scoping semantics from Scheme, which allows objects to be defined in predetermined blocks rather than the entirety of the code. S was created by Rick Becker, John Chambers, Doug Dunn, Jean McRae, and Judy Schilling at Bell Labs around 1976. Designed for statistical analysis, the language is an interpreted language whose code could be directly run without a compiler. Many programs written for S run unaltered in R. As a dialect of the Lisp language, Scheme was created by Gerald J. Sussman and Guy L. Steele Jr. at MIT around 1975.



VIT[®]
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)
CHENNAI

In 1991, statisticians Ross Ihaka and Robert Gentleman at the University of Auckland, New Zealand, embarked on an S implementation. It was named partly after the first names of the first two R authors, and partly as a play on the name of S. They began publicizing it on the data archive StatLib and the s-news mailing list in August 1993. In 1995, statistician Martin Mächler convinced Ihaka and Gentleman to make R a free and open-source software under the GNU General Public License. The first official release came in June 1995. The first official "stable beta" version (v1.0) was released on 29 February 2000.

The Comprehensive R Archive Network (CRAN) was officially announced on 23 April 1997. CRAN stores R's executable files, source code, documentation, as well as packages contributed by users. CRAN originally had three mirrors and 12 contributed packages. As of January 2022, it has 101 mirrors and 18,728 contributed packages.

The R Core Team was formed in 1997 to develop the language further. As of January 2022, it consists of Chambers, Gentleman, Ihaka, and Mächler, plus statisticians Douglas Bates, Peter Dalgaard, Kurt Hornik, Michael Lawrence, Friedrich Leisch, Uwe Ligges, Thomas Lumley, Sebastian Meyer, Paul Murrell, Martyn Plummer, Brian Ripley, Deepayan Sarkar, Duncan Temple Lang, Luke Tierney, and Simon Urbanek, as well as computer scientist Tomas Kalibera. Stefano Iacus, Guido Masarotto, Heiner Schwarte, Seth Falcon, Martin Morgan, and Duncan Murdoch were members. In April 2003, the R Foundation was founded as a non-profit organization to provide further support for the R project.

1.6. Basic Syntax in R

There are two ways to write code in RStudio:- first, in the command prompt and, second, in the R script file.

Hello World Program in R

- Launch RStudio
- In the command prompt window, type the following and press Enter

```
print("Hello, World!")
```

Output:

```
[1] "Hello, World!"
```

R Programs are usually written in R scripts and then executed in the console window

- Create a new R script file
- Write the following code, select the block of code you want to run, and then press Ctrl+Enter or click on 'Run'

```
mystring = "Hello, World!"
```

```
print(mystring)
```

Output:



```
[1] "Hello, World!"
```

1.7. Variables in R Programming

A variable is a name given to a memory location, which is used to store values in a computer program. Variables in R programming can be used to store numbers (real and complex), words, matrices, and even tables.

Variables in R Programming:

- **Declaration:**
 - Variables are created using the assignment operator (`<-` or `=`).
 - Example: `x <- 10`
- **Data Types:**
 - R is dynamically typed; you don't need to declare the data type explicitly.
 - Common types: numeric, character, logical.
 - Example: `name <- "John", age <- 25, is_student <- TRUE`
- **Variable Naming Rules:**
 - Case-sensitive, can start with a letter or a period.
 - Subsequent characters: letters, numbers, periods.
 - Example: `my_variable <- 42, .another_variable <- "Hello"`
- **Modification:**
 - Values can be changed by assigning a new value.
 - Example: `x <- 5; x <- x + 3 # Now, x is 8`

Constants in R Programming:

- **No Strict Constants:**
 - R doesn't have a strict concept of constants like some languages.
 - Conventional approach: Use uppercase names for variables not intended to change.
 - Example: `PI <- 3.14159, GRAVITY <- 9.8`

1.8. Reserved Keywords in R

Reserved keywords in R are specific words that have predefined meanings and functionalities within the language. They cannot be used as identifiers such as variable or function names. Here are some key reserved keywords in R:

- **if, else:** Used for conditional statements. **if** is used to execute code based on a condition, and **else** is used for an alternative block of code.
- **while:** Used for creating loops that continue as long as a specified condition is true.
- **for:** Used for creating loops with a known number of iterations, iterating over a sequence.



- **repeat, break, next:** **repeat** creates an infinite loop; **break** exits a loop prematurely; **next** skips the rest of the current iteration.
- **function:** Used to define functions, encapsulating a block of code for reuse.
- **return:** Used to exit a function and return a value.
- **NULL:** Represents the absence of a value or undefined value.
- **TRUE and FALSE:** Logical values representing true and false, respectively.
- **Inf:** Represents positive infinity.
- **NaN:** Represents a 'Not a Number' value, often resulting from undefined mathematical operations.

1.9.Data Types in R Programming

R has several basic data types used to define the nature of values:

1. **Numeric:** Represents decimal values, e.g., **4.5**.
2. **Integer:** For whole numbers, often specified with an **L** suffix, e.g., **4L**.
3. **Character:** Holds text/string values, e.g., **"Hello"**.
4. **Logical:** Contains Boolean values (**TRUE** or **FALSE**).
5. **Complex:** For complex numbers, e.g., **1 + 4i**.
6. **Raw:** To hold raw bytes.

1.10.R Operators

An Operator is a symbol that tells to perform different operations between operands. R programming is very rich in built-in operators. R has the following data operators:

- Arithmetic Operators
- Assignment Operators
- Logical Operators
- Relational Operators

R uses various operators for calculations, logical operations, and data manipulation.

- **Arithmetic Operators:**
 - + (addition), - (subtraction), * (multiplication), / (division), ^ (exponentiation).
 - Example: **5 + 3** results in **8**.



- **Relational Operators:**

- $>$ (greater than), $<$ (less than), $==$ (equal to), $!=$ (not equal to), $>=$ (greater than or equal to), $<=$ (less than or equal to).
- Example: $5 > 3$ evaluates to **TRUE**.

- **Logical Operators:**

- $\&$ (and), $|$ (or), $!$ (not).
- Example: **TRUE** $\&$ **FALSE** results in **FALSE**.

- **Assignment Operators:**

- $<-$ or $=$ for assigning values to objects.
- Example: $x <- 10$ assigns the value **10** to x .

Usage in Data Analysis

Operators are fundamental in R for performing various tasks:

- **Arithmetic Operations:** Used in mathematical calculations for data analysis.
- **Data Comparison:** Relational and logical operators are used to filter data or compare values.
- **Assigning Values:** Essential for creating variables and storing data.

1.11.Data Structures in R Programming

Data in R can be organized in various structures based on dimensionality and homogeneity:

- **Vectors:**

- **Definition:** A basic data structure consisting of a sequence of elements of the same data type.
- **Creation:** Using the **c()** function.
- **Example:** `numbers <- c(1, 2, 3, 4, 5)`

- **Lists:**



- **Definition:** An ordered collection of objects (elements) that can be of different data types.
- **Creation:** Using the **list()** function.
- **Example:** `my_list <- list(name="John", age=25, is_student=TRUE)`
- **Matrices:**
 - **Definition:** A 2-dimensional data structure with rows and columns of the same data type.
 - **Creation:** Using the **matrix()** function.
 - **Example:** `my_matrix <- matrix(c(1, 2, 3, 4), nrow=2, ncol=2)`
- **Data Frames:**
 - **Definition:** A 2-dimensional tabular data structure with rows and columns, similar to a spreadsheet or SQL table.
 - **Creation:** Using the **data.frame()** function.
 - **Example:** `my_df <- data.frame(name=c("John", "Jane"), age=c(25, 30), is_student=c(TRUE, FALSE))`
- **Factors:**
 - **Definition:** Used to represent categorical data. A factor consists of a set of predefined values known as levels.
 - **Creation:** Using the **factor()** function.
 - **Example:** `my_factor <- factor(c("Red", "Green", "Blue"))`
- **Arrays:**
 - **Definition:** Multi-dimensional generalization of vectors and matrices.
 - **Creation:** Using the **array()** function.
 - **Example:** `my_array <- array(c(1, 2, 3, 4), dim=c(2, 2))`
- **Data Tables (from the data.table package):**



- **Definition:** Extension of data frames with additional features for efficient data manipulation.
- **Example:** Requires installation of the data.table package.
- **Lists of Lists:**
 - **Definition:** A list where each element can be another list.
 - **Example:** `nested_list <- list(list(1, 2), list("a", "b"))`

Checking and Converting Data Types

- **Check Type:** Use functions like `is.numeric()`, `is.character()` to check the type of data.
- **Convert Type:** Convert data types with functions like `as.numeric()`, `as.character()`.

Importance of Data Types and Structures

Understanding and correctly using data types and structures is vital for effective data manipulation and analysis in R. It ensures functions and operations are applied correctly and results are accurate.

1.12.Data Frame

- **Definition:**
 - A data frame is a 2-dimensional tabular data structure in R, similar to a spreadsheet or SQL table.
 - It consists of rows and columns, where each column can be of a different data type.
- **Creation:**
 - Created using the `data.frame()` function.
 - Example:

```
my_df <- data.frame(
  name = c("John", "Jane"),
  age = c(25, 30),
  is_student = c(TRUE, FALSE) )
```
- **Characteristics:**
 - Columns have names, and rows can be indexed.
 - Columns can contain numeric, character, logical, or factor data types.
- **Accessing Data:**
 - Use column names to access specific columns.



- Example:

```
# Accessing the 'name' column
```

```
names_column <- my_df$name
```

- **Manipulation:**

- Add or modify columns easily.
- Example:

```
# Adding a new column
```

```
my_df$city <- c("New York", "San Francisco")
```

```
# Modifying an existing column
```

```
my_df$age <- my_df$age + 5
```

- **Subset and Filter:**

- Select specific rows or columns based on conditions.
- Example:
Selecting rows where age is greater than 25
subset_df <- my_df[my_df\$age > 25,]

- **Functions for Data Frames:**

- Various functions available for data frame manipulation: **summary()**, **str()**, **head()**, **tail()**, etc.

- **Integration with Other Data Structures:**

- Data frames can contain lists or factors as columns, making them versatile for diverse data types.

- **Import and Export:**

- Read data into data frames from external sources (e.g., CSV files) using functions like **read.csv()**.
- Export data frames to external files using functions like **write.csv()**.

1.13.Arrays

- An array is a multi-dimensional data structure in R that can hold elements of the same data type.
- It can have one or more dimensions.
- **Creation:**
 - Created using the **array()** function.
 - Example: `my_array <- array(c(1, 2, 3, 4), dim = c(2, 2))`
- **Attributes:**
 - Arrays have attributes specifying their dimensions (rows, columns, and more).
 - Example: `dim(my_array)` # Returns dimensions
- **Accessing Elements:**



VIT[®]
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)
CHENNAI

- Access elements using indices for each dimension.
- Example: # Accessing element in the first row and second column

```
element <- my_array[1, 2]
```

- **Multi-Dimensionality:**
 - Arrays can have more than two dimensions.
 - Example: `three_d_array <- array(1:24, dim = c(2, 3, 4))`
- **Functions for Arrays:**
 - Functions like **dim()**, **length()**, and **apply()** are commonly used for array manipulation.
- **Integration with Matrices:**
 - A matrix is a 2-dimensional array; thus, matrices are a special case of arrays in R.
- **Mathematical Operations:**
 - Supports element-wise mathematical operations and functions.
 - Example # Element-wise addition

```
result_array <- my_array + 10
```

1.14.Decision Making

Decision making is a prime feature of any programming language. It allows us to make a decision, based on the result of a condition. Decision making is involved in order to change the sequence of the execution of statements, depending upon certain conditions. A set of statements is provided to the program, along with a condition.

if-else Statements:

- Definition: Used for conditional execution. If the specified condition is true, the code inside the 'if' block is executed; otherwise, the code inside the 'else' block is executed.
- Syntax:

```
if (condition) {  
  # Code to execute if condition is TRUE  
} else {  
  # Code to execute if condition is FALSE  
}
```

Example

```
x <- 10  
if (x > 5) {  
  print("x is greater than 5")  
} else {
```



```
print("x is not greater than 5")
}
```

elseif Statement:

- Definition: Allows checking multiple conditions sequentially. Each 'elseif' block is evaluated only if the previous conditions are false.

- Syntax:

```
if (condition1) {
    # Code to execute if condition1 is TRUE
} else if (condition2) {
    # Code to execute if condition2 is TRUE
} else {
    # Code to execute if all conditions are FALSE
}
```

Example:

```
y <- 7
if (y > 10) {
    print("y is greater than 10")
} else if (y > 5) {
    print("y is greater than 5 but not 10")
} else {
    print("y is not greater than 5")
}
```

Switch Statement:

Definition: Used for multi-way branching. It selects one of several blocks of code to execute based on the value of an expression.

Syntax:

```
switch(expression, case1, case2, ..., default)
```

Example:

```
day <- "Wednesday"
switch(day,
    "Monday" = print("Start of the week"),
    "Friday" = print("End of the week"),
    print("Midweek")
)
```



The function of a looping statement is to execute a block of code, several times and to provide various control structures that allow for more complicated execution paths than a usual sequential execution.

The types of loops in R are as follows:

While Loop

Executes a block of code repeatedly as long as a specified condition is true.

Syntax:

```
while (condition) {  
  # Code to execute while condition is TRUE  
}
```

Example:

```
i <- 1  
while (i <= 5) {  
  print(i)  
  i <- i + 1  
}
```

Explanation: In this example, the code inside the while loop will be executed as long as the condition $i \leq 5$ is true. It prints the value of i and increments it by 1 in each iteration.

For Loop

For loop is one of the control statements in R programming that executes a set of statements in a loop for a specific number of times, as per the vector provided to it.

Syntax:

```
for (variable in sequence) {  
  # Code to execute for each iteration  
}
```

Example:

```
for (i in 1:5) {  
  print(i)  
}
```

Explanation: In this example, the for loop iterates over the sequence of numbers from 1 to 5. For each iteration, it prints the value of i .

1.16. Loop-control Statements



Loop-control statements are part of control statements in R programming that are used to change the execution of a loop from its normal execution sequence. There are two loop-control statements in R

Break Statement

The break statement is used to terminate the execution of a loop prematurely. When encountered, it immediately exits the loop, and the program continues with the next statement after the loop.

Syntax:

```
for (variable in sequence) {  
  # Code to execute for each iteration  
  if (condition) {  
    break  
  }  
}
```

For example:

```
for (i in 1:10) {  
  if (i == 5) {  
    break  
  }  
  print(i)  
}
```

Explanation: In this example, the loop will print values from 1 to 4. When i becomes 5, the break statement is encountered, and the loop is terminated.

Next Statement

The next statement is used to skip the rest of the code inside a loop for the current iteration and move on to the next iteration.

Syntax:

```
for (variable in sequence) {  
  # Code to execute for each iteration  
  if (condition) {  
    next  
  }  
  # Code following 'next' will be skipped if the condition is true  
}
```



Example:

```
for (i in 1:5) {  
  if (i == 3) {  
    next  
  }  
  print(i)  
}
```

Explanation: In this example, when i is equal to 3, the next statement is encountered. The code following next for that iteration is skipped, so the number 3 is not printed.

1.17.Functions in R

1. R Functions Types:

- **Definition:** A function in R is a reusable block of code designed to perform a specific task. There are two main types of functions: in-built functions provided by R and user-defined functions created by the programmer.

2. In-built Functions:

- **Definition:** In-built functions are functions that come pre-defined in the R programming language. They serve various purposes and are ready to use without the need for the programmer to implement their logic.
- **Syntax:**

```
result <- inbuilt_function(arg1, arg2, ...)
```

- **Example:**

```
# Using the sum() in-built function
```

```
numbers <- c(1, 2, 3, 4, 5)
```

```
total <- sum(numbers) print(total)
```

- **Explanation:** The **sum()** function is an in-built function that calculates the sum of a vector of numbers. In this example, it adds the numbers 1 through 5.

Common In-built Functions:

- **sum():** Computes the sum of a vector of numbers.
- **mean():** Calculates the mean (average) of a set of values.
- **sqrt():** Computes the square root of a number.
- **length():** Returns the number of elements in a vector or list.



- `max()`, `min()`: Find the maximum or minimum value in a set of numbers.

3. User-defined Functions:

- **Definition:** User-defined functions are created by the programmer to encapsulate a set of operations into a single block of code. These functions can be customized to perform specific tasks as needed.

- **Syntax:**

```
function_name <- function(arg1, arg2, ...)  
{  
  # Code block to perform operations  
  return(result)  
}
```

- **Example:**

```
# User-defined function to calculate the square of a number  
square <- function(x) {  
  result <- x^2  
  return(result)  
}  
# Using the user-defined function  
result <- square(4)  
print(result)
```

- **Explanation:** The **square** function takes a single argument **x** and returns the square of that number. In this example, it calculates and prints the square of 4.

1.18.Importing and Exporting Data in R

The ability to import and export data is a key feature in R, allowing users to work with a variety of data formats. This flexibility is crucial for data analysis, as data often comes in different formats and needs to be exported for use in different applications or for reporting. File formats like CSV, XML, xlsx, JSON, and web data can be imported into the R environment to read the data and perform data analysis, and also the data present in the R environment can be stored in external files in the same file formats.

Importing Data

1. From CSV Files:

- Use **read.csv()** for reading comma-separated values (CSV) files.
- Example: **data <- read.csv("path/to/file.csv")**.

2. From Excel Files:



VIT[®]
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)
CHENNAI

- The **readxl** package provides functions like **read_excel()** to import Excel files.
- Example: **install.packages("readxl"); library(readxl); data <- read_excel("path/to/file.xlsx")**.

3. From Databases:

- Packages like **DBI** and **RMySQL** (for MySQL), **RSQLite** (for SQLite) enable connection to databases.
- Example: Establish a connection and use **dbReadTable()** to read tables.

4. Other Formats:

- R can import other formats like JSON, XML, and SPSS files using packages like **jsonlite**, **XML**, and **haven**.

Exporting Data

1. To CSV Files:

- Use **write.csv()** to write a data frame to a CSV file.
- Example: **write.csv(data, "path/to/newfile.csv")**.

2. To Excel Files:

- The **writexl** package allows exporting data frames to Excel.
- Example: **install.packages("writexl"); library(writexl); write_xlsx(data, "path/to/newfile.xlsx")**.

3. To Databases:

- Similar to importing, use **dbWriteTable()** from appropriate database packages to export data to a database.

4. Other Formats:

- Convert and save data in various formats like JSON, XML, etc., using corresponding packages.

Considerations for Importing and Exporting Data



- **Data Type Preservation:** Ensure that data types are correctly preserved during import and export.
- **Handling Large Datasets:** For very large datasets, functions like **fread()** from **data.table** package can be more efficient.
- **Path Specification:** Be mindful of the correct file path and permissions when reading from or writing to files.

Importing and exporting data efficiently in R is essential for integrating R into broader data processing and analysis workflows.

1.19.Data Visualization

Data visualization is a critical aspect of exploratory data analysis, providing a graphical representation of data to uncover patterns, trends, and insights that might not be obvious from raw data.

Basic Principles of Data Visualization

- **Clarity and Simplicity:** Visualizations should be easy to understand and interpret.
- **Appropriate Chart Types:** Choose the right type of chart based on the data and the story you want to tell.
- **Labeling and Legends:** Proper labels and legends are essential for clarity.
- **Color Usage:** Use colors effectively to enhance understanding, not to confuse or mislead.

Introduction to GGplot2

ggplot2 is a widely-used R package for creating advanced visualizations. It's based on the Grammar of Graphics, a concept that provides a deep and flexible foundation for creating graphics.

- **Syntax:** **ggplot2** uses a layering syntax, starting with a base layer that specifies the dataset and the variables to plot, and adding layers for aesthetics, geometries, stats, scales, and more.
- **Aesthetics and Geometries:** Customize plots by setting aesthetics like color, size, shape, and by adding different types of geometric objects like lines, points, and bars.

Creating Basic Plots

- **Scatter Plots:** Useful for examining the relationship between two continuous variables.



VIT[®]
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)
CHENNAI

- **Bar Charts:** Ideal for comparing quantities across different categories.
- **Line Graphs:** Best for showing trends over time.

Advanced Plotting

- **Customizing Plots:** Modify themes, fonts, colors, and layouts to create more customized visualizations.
- **Faceting:** Create multi-panel plots by splitting the data into subsets.
- **Interactive Visualizations:** Use packages like **plotly** or **shiny** for interactive web-based visualizations.

1.20.Data visualization in R can be performed in the following ways:

- Base Graphics
- Grid Graphics
- Lattice Graphics
- ggplot2

Base Graphics:

Definition: Base graphics are the default plotting system in R, providing a simple and flexible way to create various types of plots.

Key Functions:

- `plot()`: Creates scatterplots and other basic plots.
- `hist()`: Generates histograms.
- `boxplot()`: Creates box-and-whisker plots.
- Example:

```
x <- c(1, 2, 3, 4, 5)
y <- c(2, 4, 1, 7, 3)
plot(x, y, main = "Scatterplot", xlab = "X-axis", ylab = "Y-axis")
```

Grid Graphics:

Definition: Grid graphics provide a structured approach to creating complex plots by dividing the plotting region into a grid of viewports.

Key Functions:

- `grid.newpage()`: Starts a new page for grid graphics.
- `grid.points()`: Adds points to the grid.
- `grid.lines()`: Adds lines to the grid.
- Example:
`library(grid)`



```
grid.newpage()
grid.points(x, y, pch = 16, col = "blue")
```

Lattice Graphics:

Definition: Lattice graphics, provided by the lattice package, focus on creating conditioned plots, allowing users to create multiple plots conditioned on some variable.

- Key Functions:
- `xyplot()`: Creates conditioned scatterplots.
- `histogram()`: Generates conditioned histograms.
- `bwplot()`: Creates conditioned box-and-whisker plots.

Example:

```
library(lattice)
my_data <- data.frame(x = x, y = y)
xyplot(y ~ x | cut(x, breaks = 3), data = my_data, type = c("p", "smooth"))
```

ggplot2:

Definition: ggplot2 is a powerful and popular data visualization package in R, providing a grammar of graphics for creating complex and layered visualizations.

Key Functions:

- `ggplot()`: Initializes a ggplot object.
- `geom_point()`: Adds points to the plot.
- `geom_line()`: Adds lines to the plot.
- `facet_wrap()`: Creates conditioned plots.

Example:

```
library(ggplot2)
my_data <- data.frame(x = x, y = y)
ggplot(my_data, aes(x = x, y = y)) +
  geom_point(color = "blue") +
  geom_line(color = "red") +
  labs(title = "Scatterplot with Line")
```

Advantages of Data Visualization in R:

- R offers a broad collection of visualization libraries in addition to extensive online guidance on their usage.
- R also offers data visualization in the form of 3D models and multipanel charts.
- Through R, we can easily customize our data visualization by changing axes, fonts, legends, annotations, and labels.



VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

CHENNAI

Summary

- R Programming , History, Introduction,
- Advantages, disadvantages and basics of R programming language.
- R Installation, Variables and data types, operators, decision-making structures, loops, functions and arguments in R.
- Loops, functions and arguments and objects in R programming, Data structures and data objects.
- Data Visualization in R.